



Accretion

SECURITY ASSESSMENT

Provided by Accretion Labs Pte Ltd. for Solomon Labs
August 16, 2026
A26SOL2



Solomon Labs

AUDITORS

Role	Name
Lead Auditor	Robert Reith (robert@accretion.xyz)
Senior Auditor	Mahdi Rostami (mahdi@accretion.xyz)
Assoc. Auditor	Timur Dzhandzhava (timur@accretion.xyz)

CLIENT

Solomon Labs (<https://solomonlabs.org>) engaged Accretion to conduct a security assessment of Chancery, a smart contract platform providing secure and scalable decentralized application infrastructure.

ENGAGEMENT TIMELINE

30 Jul	Project Kickoff Initial planning and scope definition
30 Jul	Assessment Begins Security review and testing phase
04 Aug	Review Fixes Security recommendations are given and implemented
16 Aug	Project Completion Report delivery and on-chain confirmation

AUDITED CODE

#	Program ID	Repository
1	ChnryP5RcZtMvP8vvVyPGUhwCg6uDJc53vCe3sxr11Sz	https://github.com/SolomonLabs/chancery-audits

ASSESSMENT

The security assessment of Solomon Labs's Solomon Labs revealed 23 findings: 0 critical, 0 high, 7 medium, 15 low, and 1 informational. All 7 medium issues were fixed, addressing risks such as unauthorized token burns, bypassed asset lifecycle controls, and flawed pause-bit semantics. Of the 15 low findings, 9 were fixed and 6 acknowledged. The single informational finding was fixed. The overall security posture is satisfactory, with no critical or high-severity vulnerabilities identified.

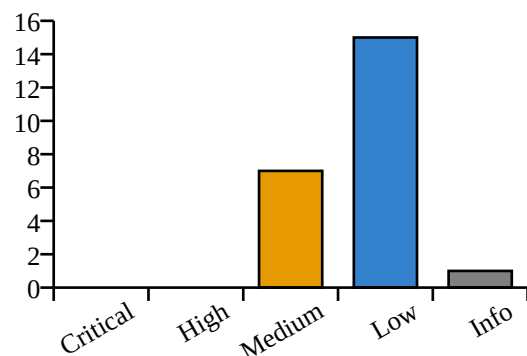
CODE ASSESSMENT

The codebase demonstrates a structured architecture with formal verification coverage and well-defined permission and governance subsystems. However, gaps in formal model coverage, unused permission bits, and several edge-case oversights in limit enforcement indicate areas where rigor should be tightened.

KEY FINDINGS

No critical or high severity findings were identified. The most significant issues were medium severity, including unauthorized burns in `redeem_trilateral`, bypassed lifecycle controls in cross-chain redemption, premature inbound message retirement, and conflated pause-bit semantics.

SEVERITY DISTRIBUTION



ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: Chancery

Link: <https://github.com/SolomonLabs/chancery-audits>

Commit: c1d461a85380029471fee0017967ef4fc3a31c21

Program ID: ChnryP5RcZtMvP8vvVyPGUhwCg6uDJc53vCe3sxr11Sz

Audit Result:

- **Audited Commit:** 59f0e4601072b9609bcaee6157931a879e8f6e31

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-M1	Unmodeled handler mints issued tokens against the formal verification coverage rule	MEDIUM	FIXED
ACC-M2	Omitted Pathway Period Caps Can Permanently Block Inbound Nonces	MEDIUM	FIXED
ACC-M3	Fee recipient key is not checked against protocol signer identities	MEDIUM	FIXED
ACC-M4	Shared Permission Pause Bit Allows Griefing of Accepted Governance Changes	MEDIUM	FIXED
ACC-M5	Missing Pathway-Kind Check Allows Premature Retirement of Valid Inbound Messages	MEDIUM	FIXED
ACC-M6	Outbound Cross-Chain Redemption Bypasses Asset Lifecycle Controls	MEDIUM	FIXED
ACC-M7	RedeemTrilateral Can Burn System-Owned or Incinerator-Owned Token Accounts to Redeem Assets	MEDIUM	FIXED
ACC-L1	Reserve destination owner is not checked against protocol signer identities	LOW	FIXED
ACC-L2	Stale Governance Approval Can Restore Revoked Permissions	LOW	FIXED
ACC-L3	Pathway-Scoped Senders Can Consume Unrelated Remote Corridor Nonces	LOW	FIXED
ACC-L4	Non-Receiveable Recipient ATA Can Block the Inbound Corridor	LOW	FIXED
ACC-L5	Product-Only Rate Validation Can Retroactively Overvalue Outstanding Supply	LOW	ACKNOWLEDGED
ACC-L6	Unequal Cross-Chain Token Decimals Can Inflate Inbound Mints	LOW	ACKNOWLEDGED
ACC-L7	Governance Key Reuse Can Resurrect Superseded Authority Transfers	LOW	ACKNOWLEDGED
ACC-L8	Short-Lived Outbound Messages Can Clog Remote Strict-Nonce Delivery	LOW	ACKNOWLEDGED
ACC-L9	Shared Party Limit Windows Can Block Valid Settlements Across Assets	LOW	ACKNOWLEDGED
ACC-L10	Fee Policies Can Be Valid but Unexecutable for Every Settlement Amount	LOW	ACKNOWLEDGED
ACC-L11	Basic Freeze Metadata Cannot Be Updated While the Token Account Remains Frozen	LOW	FIXED
ACC-L12	Activating a dormant collateral transfer hook freezes the extension observation and the asset reserve	LOW	ACKNOWLEDGED

ACC-L13	Ten role bits are classified as grantable or dangerous but no handler ever reads them	LOW	FIXED
ACC-L14	Pathway Limits Can Permit Excess Redeem Outflows by Mixing Token Denominations	LOW	FIXED
ACC-L15	Outbound Messages Can Burn Gross Amount Without a Minimum Remote Amount	LOW	FIXED
ACC-I1	Formal Solvency Model Does Not Reflect Deployed Reserve Accounting	INFO	FIXED

DETAILED ISSUES

ACC-M1 Unmodeled handler mints issued tokens against the formal verification coverage rule

MEDIUM

FIXED

Description

`reclaim_expired_outbound` mints issued tokens while being marked as not modeled in the formal verification coverage matrix. The formal verification specification permits unmodeled handlers only under an explicit aggregate-coverage argument, and states categorically that no unmodeled handler may mint or burn issued tokens or move backing. `COVERAGE.md` lists the handler as `Not modeled`, and the handler calls `cpi_mint_to`

Every other mint, burn or backing CPI site in the program belongs to a modeled instruction. The sibling `expire_inbound_message` is also not modeled but performs no token CPI, so it is correctly covered by the aggregate argument. This handler is the only exception

The effect reaches the model's own algebra. `emit_outbound_message` decrements `total_backing` in the QED specification, so an unmodeled re-mint of that same value is invisible to the `settlement_backing_conserved` obligation. The invariant can be false on chain while the obligation still discharges

The handler was introduced by the July 2026 expiry and reclaim work. The last written attestation of the mint and burn sweep predates it, and no automated check enforces the sweep: `ValidateFormalModelParity.mjs` regenerates the coverage matrix and rejects drift in what is listed, but never asserts that nothing unmodeled mints. The same specification requires that adding any handler which can touch modeled state extends the model rather than the aggregate argument, which is the case here

Location

- https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/cross_chain/instructions/reclaim_expired_outbound.rs#L660
- https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/spec/12_Formal_Verification.md#L70-L78
- https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/formal_verification/COVERAGE.md#L74-L76
- https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/formal_verification/qed/chancery_validity.qedspec#L1052-L1056
- https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/formal_verification/qed/chancery_validity.qedspec#L1080

Relevant Code

```
cpi_mint_to(  
    issued_token_program_account_info,  
    issued_token_mint_account_info,  
    recipient_issued_token_account_info,  
    mint_authority_pda_account_info,  
    mint_authority_b,  
    reclaim_amount,  
)?;
```

Mitigation Suggestion

Add `reclaim_expired_outbound` to the QED specification with its mint effect on issued supply and backing, then regenerate the coverage matrix so `settlement_backing_conserved` accounts for the reclaim mint. Extend

`ValidateFormalModelParity.mjs` to fail when any instruction marked not modeled contains a `cpi_mint_to`, `cpi_burn` or `cpi_transfer_asset_*` call site.

If modelling is deferred, record the exemption explicitly in the formal verification specification and in the coverage matrix, so the categorical rule is not contradicted by the shipped tree

Remediation

Fixed in commit: [SolomonLabs/chancery-audits#81](#)

Description

`consume_inbound_message` enforces the pathway `LimitPolicy` per-transaction cap and its hourly, daily, seven-day, and thirty-day volume caps. However, `classify_message_retirement` receives only `pathway_per_transaction_maximum`, so `expire_inbound_message` cannot classify a message that exceeds an absolute period cap as terminally unexecutable.

This gap matters when `per_transaction_maximum = 0`. `assert_per_tx` treats `0` as unbounded, and the non-decreasing cap check skips zero caps. Therefore, a pathway policy can validly pair an unbounded per-transaction cap with a finite period cap.

For example, a pathway limit policy has `per_transaction_maximum = 0` and `per_hour_maximum = 100`. Alice, a legitimate remote emitter, submits an authenticated inbound message for nonce `0` with amount `101`. `consume_inbound_message` passes the per-transaction check but fails with `HourlyLimitBreached` even against an empty hourly window. The nonce does not advance.

The same message also cannot be retired. `expire_inbound_message` sees `pathway_per_transaction_maximum = 0`, skips the pathway cap check, and returns `MessageNotExpired`. Every later nonce then fails `RemoteNonceMismatch` because nonce `0` remains the strict inbound head.

Operations can also create this condition after message emission by tightening a period cap. Restrictive cap tightening is directly executable, while widening the cap back is a widening change and is timelocked.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/expire_inbound_message.rs#L183-L245
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/expire_inbound_message.rs#L304-L328
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/expire_inbound_message.rs#L357-L367
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/consume_inbound_message.rs#L548-L634
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/limits/state/limit_policy.rs#L264-L275
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/limits/state/limit_policy.rs#L364-L368
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/limits/state/limit_policy.rs#L397-L402

Relevant Code

```
pub(crate) fn classify_message_retirement(
    message_kind:      u8,
    expires_at_unix_timestamp: i64,
    amount:             u128,
    now:                i64,
    message_expiry_seconds: u64,
    per_message_maximum:   u64,
    per_day_maximum:       u64,
    remote_asset_binding_valid: bool,
    pathway_per_transaction_maximum: u64,
) -> Result<u8, ChanceryError> {
```

```

if pathway_per_transaction_maximum != 0
  && amount > pathway_per_transaction_maximum as u128
{
  return Ok(
    inbound_message_retirement_reason::PATHWAY_PER_TRANSACTION_CAP_TIGHTENED,
  );
}

```

```

limit_policy.per_transaction_maximum

```

```

limit_policy.assert_per_tx(args.amount)?;

```

```

limit_policy.assert_window_volume(
  hourly_usage_window.gross_in,
  args.amount,
  ChanceryError::HourlyLimitBreached,
)?;

```

```

for cap in [
  self.per_transaction_maximum,
  self.per_hour_maximum,
  self.per_day_maximum,
  self.per_seven_day_maximum,
  self.per_thirty_day_maximum,
] {
  if cap == 0 {
    continue;
  }
}

```

```

pub fn assert_per_tx(&self, amount: u64) -> Result<(), ProgramError> {
  if self.per_transaction_maximum != 0 && amount > self.per_transaction_maximum {
    return Err(ChanceryError::PerTxLimitBreached.into());
  }
}

```

```

let new_total: u128 = window_gross
  .checked_add(additional as u128)
  .ok_or(ChanceryError::ArithmeticOverflow)?;

if new_total > cap as u128 {
  return Err(error.into());
}

```

Mitigation Suggestion

Include pathway period caps in terminal retirement classification. As an immediate configuration mitigation, require every pathway limit policy to set a non-zero `per_transaction_maximum` whenever any period cap is set.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/78>.

Description

We found that `fee_recipient_key` is the only one of the five identity surfaces named in [spec/16 §16.4](#) that is not guarded against Chancery's own signer-capable PDAs. The specification states that mint, freeze, reserve, close-mint, transfer-hook, permanent-delegate, metadata-pointer, metadata-update, pause, confidential-transfer, default-account-state and event-authority PDAs may not be assigned as permission subjects, governance or operations identities, principals, executors, or external fee recipients

The first four surfaces are enforced by `assert_external_identity` and `assert_optional_external_identity`, which reject any key matching one of the twelve protocol-signer derivations. Permission subjects are guarded in `permissions/mutation.rs` and `permissions/auth.rs`, governance identities in `propose_authority_transfer.rs`, and principals and executors in `create_settlement_intent.rs`, `settlement_policy.rs` and `pathway_policy.rs`

`FeePolicy::assert_parameter_sanity` constrains `fee_recipient_key` only for zero and nonzero coherence against the routing mode. Which nonzero key is acceptable is never constrained, so `register_fee_policy` – operations-authority-only and immediate – accepts a routed policy naming a protocol PDA, and settlement routes the net fee to that PDA's associated token account on every mint and redeem through the referencing pathway

No handler constructs a transfer authorised by the mint, freeze or event authority, and `reserve_authority` signs only transfers out of the canonical per-asset reserve account, so fees accrued at such an address cannot be moved without a program upgrade. Rebinding `fee_policy_id` on the pathway is classified **Widening**, so stopping further accrual is itself timelocked

The regression test mandated by [spec/16 §16.13](#), which asserts that every protocol signer PDA is rejected as an attempted external identity, does not cover this surface

Location

https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/fees/state/fee_policy.rs#L336-L346

https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/settlement/state/settlement_policy.rs#L213-L219

https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/pathway/state/pathway_policy.rs#L399

<https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/permissions/mutation.rs#L43>

https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/core/instructions/propose_authority_transfer.rs#L101

Relevant Code

```
// NONE and RESERVE_RETENTION are non-routing policies. A recipient key
// on either mode is incoherent and risks callers assuming that an
// unvalidated account may receive the fee. Routed policies must name a
// concrete recipient owner up front.
if self.retains_fee() {
    if self.fee_recipient_key != Pubkey::default() {
        return Err(ChanceryError::FeePolicyInvalidParameters.into());
    }
} else if self.fee_recipient_key == Pubkey::default() {
```

```
        return Err(ChanceryError::FeePolicyInvalidParameters.into());
    }

    let program_id = crate::id();
    assert_optional_external_identity(&self.allowed_principal_a, &program_id)?;
    assert_optional_external_identity(&self.allowed_principal_b, &program_id)?;
    assert_optional_external_identity(&self.designated_executor, &program_id)?;

    assert_external_identity(&proposed.subject, &crate::id())?;
```

Mitigation Suggestion

Call `assert_external_identity` on `fee_recipient_key` in the routed branch of `assert_parameter_sanity`. Because that function runs at registration, on both update paths and again at consumption, this closes the surface for already-persisted policies as well as new ones. Extend the §16.13 protocol-signer-identity test to iterate `fee_recipient_key` alongside the other four surfaces so the fifth is held by a test rather than by review

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/77>.

Description

`PERMISSION_FLAG_PAUSED` is used by two independent control paths with incompatible rules. Permission upserts treat the bit as part of the semantic permission value, because `PermissionValue::to_payload` includes `permission_flags` in the pending-change hash. Clearing the bit is classified as `UNPAUSED`, which is `Widening` or `Dangerous` depending on the remaining roles, so the normal permission-update path requires governance and a timelock.

However, `set_executor_pause` and `set_counterparty_pause` can clear the same bit on the same `PermissionRecord` immediately with an operations signature. The direct unpause bypass is narrow because the endpoints only apply to pathway-scoped settlement roles, but it still gives the same bit two owners.

The larger issue is pending-change griefing. `upsert_permission_with_pending_change` recomputes the old value hash from the live `PermissionRecord` before calling `assert_accepted_pending_change`. If operations toggles `PERMISSION_FLAG_PAUSED` after governance accepts the pending change but before it is consumed, the live hash no longer matches the accepted `old_value_hash`. The recomputed risk can also differ because the pause transition changes the permission change mask. This can invalidate an accepted permission change after the full timelock has already elapsed, and the process can be repeated.

The pathway pause design avoids this problem by masking `PATHWAY_PAUSE` out of the pathway config-change payload, leaving the operational pause endpoint as the only writer for that bit. The permission path does not apply the same separation.

The root cause is bit reuse. `PERMISSION_FLAG_PAUSED` acts both as an operational containment flag and as part of permission generation identity, especially because revoke leaves a paused tombstone. Move generation or revocation identity into reserved space, then remove the operational pause flag from the hashed permission payload.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/permissions/change_detection.rs#L81-L90
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/permissions/change_detection.rs#L176-L192
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/control/pending_change.rs#L81-L87
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/permissions/instructions/upsert_permission_with_pending_change.rs#L202-L230
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/control/pending_change.rs#L134-L147
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/control/instructions/set_executor_pause.rs#L99-L109
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/control/instructions/set_counterparty_pause.rs#L101-L111
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/pathway/state/pathway_policy.rs#L160-L168
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/permissions/instructions/revoke_permission.rs#L109-L129
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/permissions/state/permission_record.rs#L94-L98

Relevant Code

```
payload[81..89].copy_from_slice(&self.permission_flags.to_le_bytes());
```

```
(true, false) => {
    change_mask |= permission_change_mask::UNPAUSED;
    raise_risk(
        &mut risk,
        if proposed_has_dangerous_roles {
            ConfigChangeRiskClass::Dangerous
        } else {
            ConfigChangeRiskClass::Widening
        },
    );
}
```

```
if args.is_paused {
    record.permission_flags |= PERMISSION_FLAG_PAUSED;
} else {
    record.permission_flags &= !PERMISSION_FLAG_PAUSED;
}
```

```
let old_hash = compute_config_change_hash(
    change_kind::UPSERT_PERMISSION,
    permission_record_account_info.key,
    classification.risk.as_u8(),
    &old_value.to_payload(),
);
```

```
if pending.risk_class != expected_risk.as_u8() {
    return Err(ChanceryError::ConfigChangeRiskClassMismatch.into());
}
```

```
if &pending.old_value_hash != expected_old {
    return Err(ChanceryError::ConfigChangeHashMismatch.into());
}
```

```
normalized.status_flags &= !status_flag::PATHWAY_PAUSE;
```

```
pub _reserved: [u8; 32],
```

Mitigation Suggestion

Separate operational pause state from permission generation identity. Store generation or revocation identity in the existing reserved bytes, then remove **PERMISSION_FLAG_PAUSED** from the hashed permission payload.

Keep **set_executor_pause** and **set_counterparty_pause** as the only direct writers for the operational pause bit, matching the pathway pause design.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/70>.

Description

`expire_inbound_message` retires an authenticated inbound head message when it is permanently unexecutable. One terminal condition is that the message amount exceeds the pathway's current `per_transaction_maximum`.

The handler verifies that the supplied `pathway_id` resolves to a canonical `PathwayPolicy` PDA, but it does not verify that the pathway is a `CROSS_CHAIN_MINT` pathway. Because the message hash does not include `pathway_id` or `pathway_kind`, a `CROSS_CHAIN_REDEEM` sibling over the same asset and issued-token mint can reconstruct the same message hash and validate the same signatures.

`consume_inbound_message` correctly requires `CROSS_CHAIN_MINT`, but `expire_inbound_message` can use the redeem sibling's limit policy. If the intended mint pathway allows amount `100` and the redeem sibling caps at `50`, the message can be retired as `PATHWAY_PER_TRANSACTION_CAP_TIGHTENED` even though it remains consumable under the intended mint pathway.

This advances the inbound nonce and permanently retires the message. The recipient does not receive the issued tokens, and the emitted retirement evidence can enable the source-side reclaim flow. This is a denial-of-service and evidence-integrity issue rather than direct theft.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/expire_inbound_message.rs#L279-L315
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/expire_inbound_message.rs#L343-L353
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/consume_inbound_message.rs#L312-L319
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/message_hash.rs#L42-L45

Relevant Code

```
// ■■ Pathway: canonical PDA only (identity for preimage fields) ■■■■■■■■■■■■
let pathway_policy_bump = PathwayPolicy::verify_pda(pathway_policy_account_info, &args.pathway_id, &program_id)?;
let pathway = PathwayPolicy::load_for_verified_pda(
    pathway_policy_account_info,
    &args.pathway_id,
    pathway_policy_bump,
)?;
```

```
limit_policy.per_transaction_maximum
```

```
let retirement_reason = classify_message_retirement(
    args.message_kind,
    args.expires_at_unix_timestamp,
    args.amount,
    clock.unix_timestamp,
    remote_policy.message_expiry_seconds,
    remote_policy.per_message_maximum,
    remote_policy.per_day_maximum,
    remote_asset_binding_valid,
    pathway_per_transaction_maximum,
)?;
```

```
pathway.assert_active()?;  
pathway.assert_kind(pathway_kind::CROSS_CHAIN_MINT)?;
```

```
//! `intent_id` and `pathway_id` are intentionally NOT in the preimage –  
//! they are local-only concepts that the remote chain has no ability to  
//! compute. Local audit binding happens in the canonical evidence events  
//! after the hash is verified.
```

Mitigation Suggestion

Require the expiry pathway to be the inbound mint pathway immediately after loading it:

```
pathway.assert_kind(pathway_kind::CROSS_CHAIN_MINT)?;
```

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/58>.

Description

`emit_outbound_message` implements outbound cross-chain redemption. It burns the sender's issued tokens and commits an authenticated message that authorizes a remote token mint or asset release.

The instruction checks the global `REDEEM` pause, `AssetPauseState`, the cross-chain pathway, sender permission, issued-token controls, remote-domain policy, signer set, and configured limits. However, it never loads the canonical `AssetConfig` for `pathway.asset_mint`.

As a result, a `FROZEN` asset can still be redeemed through an active outbound cross-chain corridor if the separate pause controls remain open. This bypasses the asset lifecycle control that blocks local redemption.

This is a containment issue, not an independent solvency issue. The sender still burns issued tokens, and no local collateral is transferred by `emit_outbound_message`. However, `AssetConfig.mode` governs user settlement eligibility, and outbound cross-chain redemption is still a user settlement path.

The omission is inconsistent with local redeem handlers, which enforce `AssetConfig` redemption checks, and with `consume_inbound_message`, which loads `AssetConfig` for the mirror cross-chain settlement path.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L110-L136
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L261-L306
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L694-L720
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/consume_inbound_message.rs#L342-L360
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_direct.rs#L287-L294
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/core/state/asset_config.rs#L303-L307
- [https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/spec/06_Reserve Compartments Provenance and Enforcement.md#L71](https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/spec/06_Reserve%20Compartment%20Provenance%20and%20Enforcement.md#L71)

Relevant Code

```
const ASSET_PAUSE_STATE:          usize = 3;
const PATHWAY_POLICY:             usize = 4;
const REMOTE_DOMAIN_POLICY:       usize = 5;
const SIGNER_SET:                 usize = 6;
const REMOTE_NONCE_ACCOUNT_INDEX: usize = 7;
const SOURCE_ISSUED_TOKEN_ACCOUNT: usize = 8;
const ISSUED_TOKEN_MINT:          usize = 9;
const ISSUED_TOKEN_PROGRAM:       usize = 10;
const SENDER:                    usize = 11;
const ISSUED_TOKEN_CONTROL:       usize = 12;
const SENDER_PERMISSION_RECORD:   usize = 13;
const REQUIRED_ACCOUNT_COUNT:     usize = 14;
```

```
let pause_state_bump = PauseState::verify_pda(pause_state_account_info, &crate::id());
let pause_state = PauseState::load_for_verified_pda(pause_state_account_info, pause_state_bump);
```

```
pause_state.assert_not_paused_for(pause_bit::REDEEM, clock.slot)?;
```

```
asset_pause_state.assert_not_paused_for(pause_bit::REDEEM, clock.slot)?;
```

```
    cpi_burn(  
        issued_token_program_account_info,  
        source_issued_token_account_info,  
        issued_token_mint_account_info,  
        sender_account_info,  
        args.amount,  
    )?;
```

```
asset_config.assert_extensions_fresh(clock.slot)?;  
asset_config.assert_not_paused()?;  
asset_config.assert_deposits_permitted()?;  
asset_config.assert_extensions_approved()?;  
pathway.assert_collateral_extensions_allowed(asset_config.observed_extension_mask)?;
```

```
asset_config.assert_extensions_fresh(clock.slot)?;  
asset_config.assert_not_paused()?;  
asset_config.assert_redeems_permitted()?;  
asset_config.assert_extensions_approved()?;  
pathway.assert_collateral_extensions_allowed(asset_config.observed_extension_mask)?;
```

```
pub fn assert_redeems_permitted(&self) -> Result<(), ProgramError> {  
    match self.mode {  
        asset_mode::ACTIVE | asset_mode::WIND_DOWN => Ok(()),  
        asset_mode::FROZEN => Err(ChanceryError::AssetModeForbids.into()),  
        _ => Err(ChanceryError::AssetModeForbids.into()),  
    }  
}
```

Mitigation Suggestion

Add the canonical `AssetConfig` account to `emit_outbound_message`, derive it from `pathway.asset_mint`, and call `assert_redeems_permitted` before advancing the outbound nonce or burning issued tokens.

Apply any required freshness and extension checks consistently with the other settlement paths.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/67>.

ACC-M7 RedeemTrilateral Can Burn System-Owned or Incinerator-Owned Token Accounts to Redeem Assets

MEDIUM

FIXED

Description

The `redeem_trilateral` instruction burns `source_issued_token_account` with `principal_a` as the burn authority, but it does not verify that `source_issued_token_account` is owned by `principal_a`.

This ownership invariant is enforced in the `redeem_direct` and `redeem_delegated` instructions with `assert_spl_token_account_binding`. The trilateral path omits the same check before calling `cpi_burn`.

This is unsafe because the SPL Token burn processor skips authority validation when the source token account is owned by the system program or the incinerator. As a result, tokens held in those accounts can be burned by any caller through `redeem_trilateral`, and the burn can be used to redeem reserve assets.

Exploit scenario:

1. Alice, a permitted `principal_a`, prepares a `redeem_trilateral` settlement using a `source_issued_token_account` whose token-account owner is the system program or the incinerator. 2. Alice signs as `principal_a`. 3. The `redeem_trilateral` instruction does not check that `source_issued_token_account` is owned by Alice. 4. The SPL Token burn processor skips `validate_owner` for system-owned or incinerator-owned token accounts, so the burn succeeds. 5. The `redeem_trilateral` instruction treats the burn as Alice's redemption and transfers reserve assets to Bob, the selected `principal_b`.

Tokens sent to the system program or incinerator are intended to be burnable, but they should not be usable to redeem assets from the protocol.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_trilateral.rs#L247-L263
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_trilateral.rs#L602-L610
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_trilateral.rs#L808-L829
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_direct.rs#L341-L346
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_delegated.rs#L574-L582
- <https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/mod.rs#L455-L478>
- <https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/mod.rs#L556-L590>
- <https://github.com/solana-program/token/blob/26abb3fc1bd5f99cfe0c1bf7cc2cd9a845f6c01b/pinocchio/program/src/processor/shared/burn.rs#L62-L83>

Relevant Code

```
let source_issued_token_account_info = &accounts[SOURCE_ISSUED_TOKEN_ACCOUNT];
let reserve_asset_token_account_info = &accounts[RESERVE_ASSET_TOKEN_ACCOUNT];
let destination_asset_token_account_info = &accounts[DESTINATION_ASSET_TOKEN_ACCOUNT];
let asset_mint_account_info = &accounts[ASSET_MINT];
let issued_token_mint_account_info = &accounts[ISSUED_TOKEN_MINT];
let reserve_authority_pda_account_info = &accounts[RESERVE_AUTHORITY_PDA];
```

--

```

        &ix,
        &[account_account_info.clone(), mint_account_info.clone(), authority_account_info.clone(), token_program_account_info.clone()],
    )
}

```

```

pub fn assert_spl_token_account_binding(
    token_account_info: &AccountInfo,
    token_program_account_info: &AccountInfo,
    expected_mint: &Pubkey,
    expected_owner: &Pubkey,
) -> ProgramResult {
    if token_account_info.owner != token_program_account_info.key {
        return Err(ChanceryError::TokenProgramMismatch.into());
    }

    let data = token_account_info.try_borrow_data()?;

    if data.len() < 64 {
        return Err(ChanceryError::AccountDataLengthMismatch.into());
    }

    let mut mint_bytes: [u8; 32] = [0u8; 32];
    let mut owner_bytes: [u8; 32] = [0u8; 32];

    mint_bytes.copy_from_slice(&data[0..32]);
    owner_bytes.copy_from_slice(&data[32..64]);

    if mint_bytes != expected_mint.to_bytes() {
        return Err(ChanceryError::AccountKeyMismatch.into());
    }

    if owner_bytes != expected_owner.to_bytes() {
        return Err(ChanceryError::AccountKeyMismatch.into());
    }
}

```

```

if likely(!source_account.is_owned_by_system_program_or_incinerator()) {
    match source_account.delegate() {
        Some(delegate) if pubkey_eq(authority_info.key(), delegate) => {
            // SAFETY: `authority_info` is not currently borrowed.
            unsafe { validate_owner(delegate, authority_info, remaining)? };

            let delegated_amount = source_account
                .delegated_amount()
                .checked_sub(amount)
                .ok_or(TokenError::InsufficientFunds)?;
            source_account.set_delegated_amount(delegated_amount);

            if delegated_amount == 0 {
                source_account.clear_delegate();
            }
        }
        _ => {
            // SAFETY: `authority_info` is not currently borrowed.
            unsafe { validate_owner(&source_account.owner, authority_info, remaining)? };
        }
    }
}
}

```

Mitigation Suggestion

Call `assert_spl_token_account_binding` on `source_issued_token_account_info` before `cpi_burn` in `redeem_trilateral`, using the issued-token mint and `principal_a_account_info.key` as the expected owner.

Add a regression test where `source_issued_token_account` is owned by the system program or incinerator and verify that `redeem_trilateral` rejects it before the burn.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/issues/68>.

Description

`register_reserve_destination_with_pending` never checks that `destination_owner` is an external identity, so a token account owned by one of Chancery's own signer-capable PDAs is a legal reserve destination.

The handler validates the destination token account thoroughly in every other respect: it binds the owning token program, requires the account's mint field to equal the asset mint, and requires the account's owner field to equal the supplied `destination_owner`. What it never does is constrain which owner is acceptable. `assert_external_identity` is not called anywhere in `modules/reserve/`.

This is the same defect shape as the fee recipient key reported separately, on the largest value-outflow surface in the program. The security remediation invariants bar signer-capable Chancery PDAs from permission subjects, governance and operations identities, principals, executors and external fee recipients. A reserve destination owner is functionally the same kind of external payee and is not on that list.

Funds routed to such a destination cannot be retrieved. Every one of the four callers that signs with `reserve_authority` independently re-derives the canonical per-asset associated token account and pins the source to it, so no instruction can authorise a transfer out of a destination account owned by the mint, freeze, event or any other protocol PDA. `destination_owner` is immutable after registration, so the destination cannot be corrected in place either; it can only be disabled to stop further outflow.

Verified with a LiteSVM test at this commit: running the real governance propose, accept and consume ceremony with `destination_owner` set to the mint-authority PDA persists the destination, and a subsequent `withdraw_reserve` moves 1,250 units out of a 5,000 unit reserve into the PDA-owned account, where they remain unreachable.

The reason this is reported as Low is that the surrounding path is genuinely well guarded and the loss is bounded, observable and stoppable. Registration requires the governance authority and an accepted pending change whose payload hash commits the exact `destination_owner` bytes, and the record is written with `status = DISABLED`; enabling it requires a second, independent governance ceremony, since the `DISABLED` to `ENABLED` transition classifies `Widening` and cannot take the direct path. `assert_reserve_withdrawal_caps` rejects registration unless both a per-transaction and a daily maximum are non-zero, and both are re-enforced on every withdrawal, so the outflow is rate-limited. Every withdrawal emits `ReserveWithdrawal` naming the destination token account. The `ENABLED` to `DISABLED` transition is `RestrictiveImmediate`, so operations or the emergency authority can stop further outflow with no timelock and no pending change.

The reserve specification enumerates the destination rules exhaustively without stating this prohibition, so the omission is consistent with the written specification rather than contrary to it. What keeps the finding alive is that the barred list already extends beyond signing authorities to external fee recipients, a payee rather than a signer, so the rule's own scope contemplates payee surfaces and omits the largest one in the program

Location

- https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/reserve/instructions/register_reserve_destination_with_pending.rs#L132-L146
- https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/reserve/instructions/register_reserve_destination_with_pending.rs#L242
- https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/settlement/state/settlement_policy.rs#L213-L219
- https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/reserve/instructions/withdraw_reserve.rs#L285-L305

Relevant Code

```

if &token_data[0..32] != asset_mint_account_info.key.as_ref() {
    return Err(ChanceryError::ReserveDestinationTokenMintMismatch.into());
}

if &token_data[32..64] != args.destination_owner.as_ref() {
    return Err(ChanceryError::ReserveDestinationOwnerMismatch.into());
}

}

ReserveDestination::validate_purpose_flags(args.destination_flags)?;
// no assert_external_identity(&args.destination_owner, &crate::id())

destination.destination_owner      = args.destination_owner;

let program_id = crate::id();
assert_optional_external_identity(&self.allowed_principal_a, &program_id)?;
assert_optional_external_identity(&self.allowed_principal_b, &program_id)?;
assert_optional_external_identity(&self.designated_executor, &program_id)?;

```

Mitigation Suggestion

Call `assert_external_identity` on `args.destination_owner` in `register_reserve_destination_with_pending`, beside the existing mint and owner-field checks. The field is immutable after registration, so the check belongs there and nowhere else.

Add the reserve destination owner to the barred-identity surfaces in the security remediation invariants, and to the regression test that enforces them, whose consumer list currently covers neither the reserve nor the fees module

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/80>.

Description

Multiple identical permission changes can coexist because `proposer_nonce` is part of the `PendingConfigChange` identifier. Two proposals can therefore approve the same permission grant for the same account.

When one accepted proposal is consumed, the permission is granted. If operations later revoke that permission, the record is rewritten as a paused zero-role tombstone. The governance hash for permissions includes the subject, scope, role bits, permission flags, and expiry, but excludes provenance fields such as `issued_at_unix_timestamp`, `granted_by`, and `_reserved`.

As a result, revocation can recreate the same semantic old state that a second accepted proposal expected. That stale proposal can then pass the old-value hash check and restore the revoked permission without a fresh governance approval after the revocation.

For example, Alice has a pending accepted proposal to receive `CAN_FREEZE_TOKEN_ACCOUNT`, and a second identical accepted proposal exists with a different `proposer_nonce`. The first proposal is consumed, then operations revoke Alice's permission. The second proposal can later be consumed to restore Alice's freeze permission, allowing her to freeze a token account again.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/control/pending_change.rs#L49-L73
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/control/instructions/propose_config_change.rs#L128-L134
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/permissions/change_detection.rs#L46-L93
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/permissions/instructions/revoke_permission.rs#L109-L129
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/permissions/instructions/upsert_permission_with_pending_change.rs#L203-L231
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/permissions/instructions/upsert_permission_with_pending_change.rs#L256-L273

Relevant Code

```
pub fn compute_change_id(
    change_kind: u16,
    target_account: &Pubkey,
    new_value_hash: &[u8; 32],
    proposer: &Pubkey,
    proposer_nonce: u64,
) -> [u8; 32] {
```

```
pub struct PermissionValue {
    pub subject: Pubkey,
    pub scope_kind: u8,
    pub scope_key: Pubkey,
    pub role_bits: [u64; 2],
    pub permission_flags: u64,
    pub expiry_unix_timestamp: i64,
}
```

```
permission_record.role_bits = [0u64; 2];
permission_record.permission_flags = PERMISSION_FLAG_PAUSED;
permission_record.issued_at_unix_timestamp = clock.unix_timestamp;
permission_record.expiry_unix_timestamp = 0;
permission_record.granted_by = *revoking_authority_account_info.key;
permission_record.role_schema_version = role::PERMISSION_ROLE_SCHEMA_VERSION;
permission_record._reserved = [0u8; 32];
```

```
if &pending.old_value_hash != expected_old {
    return Err(ChanceryError::ConfigChangeHashMismatch.into());
}

if &pending.new_value_hash != expected_new {
    return Err(ChanceryError::ConfigChangeHashMismatch.into());
}
```

```
write_permission_record(
    &mut record,
    proposed,
    bump,
    clock.unix_timestamp,
    *granting_authority_account_info.key,
    rewrite_grant_provenance,
);
```

Mitigation Suggestion

Add a monotonic generation counter to **PermissionRecord**, increment it on every permission change, and include it in pending-change hashes. This prevents an accepted proposal from becoming valid again after a grant-and-revoke cycle.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/85>.

Description

`emit_outbound_message` validates a `CROSS_CHAIN_REDEEM` pathway selected by `args.pathway_id` and separately validates a remote-domain corridor selected by `args.remote_chain_kind` and `args.remote_domain_id`.

The sender permission check is scoped to the selected pathway, but the outbound nonce is consumed from the selected remote-domain corridor. The handler does not verify that the pathway's `asset_mint` is the Solana asset expected by that corridor.

The emitted cross-chain message commits `pathway.asset_mint` as `source_asset`, while the destination domain fields are taken from `RemoteDomainPolicy`. Alice, a sender authorized for pathway A, can select unrelated corridor B. The instruction burns Alice's issued tokens and consumes corridor B's next nonce, but emits a message whose `source_asset` belongs to pathway A.

If corridor B's daughter contract rejects that message due to the asset mismatch, later valid messages for corridor B use higher nonces and fail strict nonce ordering until the bad nonce is handled. This can temporarily block an unrelated corridor and lock later users' burned principal.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/pathway/state/pathway_policy.rs#L59-L75
- <https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/permissions/auth.rs#L253-L266>
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L284-L340
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L365-L399
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L422-L443
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L675-L727

Relevant Code

```
/// Collateral asset mint this pathway operates on.  
pub asset_mint: Pubkey,  
  
/// Issued token mint this pathway produces or burns.  
pub issued_token_mint: Pubkey,
```

```
assert_signer_holds_role(  
    signer_account_info,  
    permission_record_account_info,  
    role::CAN_EMIT_OUTBOUND_MESSAGE,  
    scope::PATHWAY,  
    pathway_policy,  
    program_id,  
)
```

```

let pathway_policy_bump = PathwayPolicy::verify_pda(pathway_policy_account_info, &args.pathway_id, &program_id)?;

let pathway = PathwayPolicy::load_for_verified_pda(
    pathway_policy_account_info,
    &args.pathway_id,
    pathway_policy_bump,
)?;

pathway.assert_active()?;
pathway.assert_kind(pathway_kind::CROSS_CHAIN_REDEEM)?;

```

```

let remote_policy_bump = RemoteDomainPolicy::verify_pda(
    remote_policy_account_info,
    args.remote_chain_kind,
    args.remote_domain_id,
    &program_id,
)?;

let remote_policy = RemoteDomainPolicy::load_for_verified_pda(
    remote_policy_account_info,
    args.remote_chain_kind,
    args.remote_domain_id,
    remote_policy_bump,
)?;

```

```

let source_nonce = {
    let mut n = RemoteNonce::load_mut_for_verified_pda(
        remote_nonce_account_info,
        args.remote_domain_id,
        &nonce_scope_key,
        remote_nonce_bump,
    )?;

    n.take_outbound()?
};

```

```

let source_asset_bytes = pathway.asset_mint.to_bytes();
let source_issued_bytes = pathway.issued_token_mint.to_bytes();

let preimage = MessageHashPreimage {
    source_nonce,
    source_asset: &source_asset_bytes,
    destination_asset: &args.destination_asset,
    source_issued_token: &source_issued_bytes,
    destination_issued_token: &remote_policy.remote_issued_token,
    signer_set_id: &remote_policy.signer_set_id,
    // ...
};

```

```

cpi_burn(
    issued_token_program_account_info,
    source_issued_token_account_info,
    issued_token_mint_account_info,
    sender_account_info,
    args.amount,
)?;

```

Mitigation Suggestion

Bind each **RemoteDomainPolicy** to the expected local **asset_mint** or to the allowed local pathway. Reject mismatches before calling **take_outbound** or burning issued tokens.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/83>.

Description

`consume_inbound_message` uses a strict remote-domain nonce and requires the attested recipient to receive tokens through the recipient's canonical Associated Token Account (ATA). The relay cannot choose another recipient-owned token account.

If that canonical ATA is frozen, uninitialized, missing, or otherwise unable to receive a Token-2022 mint, `consume_inbound_message` fails and the nonce does not advance. Alice, a remote user, can burn tokens with the recipient set to the owner of such an ATA. The authenticated message remains at the corridor nonce head, and later inbound messages in the same corridor cannot be consumed.

`expire_inbound_message` cannot retire this message based on recipient-account state. It can only retire it after the signed expiry lapses or after a policy condition changes enough to classify the message as terminally invalid.

This can temporarily block every later message in the same corridor and lock users' already-burned principal. Recovery requires creating or repairing the recipient ATA, privileged policy intervention, or an authorized thaw, consume, and refreeze flow.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/consume_inbound_message.rs#L448-L470
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/consume_inbound_message.rs#L675-L701
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/expire_inbound_message.rs#L189-L246
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/constants/cross_chain.rs#L51-L72

Relevant Code

```
let (expected_recipient_token_account, _) = Pubkey::find_program_address(
    &[
        recipient_owner.as_ref(),
        issued_token_program_account_info.key.as_ref(),
        pathway.issued_token_mint.as_ref(),
    ],
    &programs::ASSOCIATED_TOKEN_PROGRAM_ID,
);

if recipient_issued_token_account_info.key != &expected_recipient_token_account {
    return Err(ChanceryError::AccountKeyMismatch.into());
}
```

```
n.bump_inbound()?;

n.last_consumed_message_hash = message_hash;
```

```
cpi_mint_to(
    issued_token_program_account_info,
    issued_token_mint_account_info,
    recipient_issued_token_account_info,
    mint_authority_pda_account_info,
    mint_authority_b,
```

```
args.amount,  
)?;
```

```
Err(ChanceryError::MessageNotExpired)
```

Mitigation Suggestion

Add a safe retirement reason for canonical recipient accounts that cannot receive tokens. Alternatively, route inbound value through a claim or escrow mechanism so strict nonce advancement is not coupled to the recipient ATA's mutable state.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/84>.

Description

`assert_economic_invariants` only checks that both rates are nonzero and that `deposit_rate_e9 * redeem_rate_e9 <= 1e18`. This protects a same-rate round trip, but it does not protect issued tokens that were already minted under earlier rates.

`UpdateAssetConfigArgs` can update both `deposit_rate_e9` and `redeem_rate_e9`, and validation only checks the newly proposed pair. Since `compute_redeem_output` applies the current `redeem_rate_e9` to every burn, increasing the redeem rate retroactively revalues the full outstanding issued-token supply.

For example, assume rates are initially `(1.0, 1.0)`. Alice and Bob each deposit 1 asset and receive 1 issued token. The reserve has 2 assets and the supply is 2 issued tokens. Governance then updates the rates to `(0.5, 2.0)`, which passes the product check. Alice redeems 1 issued token and receives 2 assets, draining the reserve while only half of the supply is burned. Bob's remaining issued token is then unbacked until the reserve is recapitalized.

The formal model states the invariant per leg: mint output must not exceed the deposited asset amount, and redeem output must not exceed the burned issued amount. The implementation instead enforces only the product form, so it accepts rate pairs that the modeled invariant rejects.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/core/state/asset_config.rs#L491-L517
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/core/state/asset_config.rs#L533-L542
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/core/instructions/update_asset_config.rs#L58-L85
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/core/instructions/update_asset_config.rs#L130-L135
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/formal_verification/Spec.lean#L238-L244
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/formal_verification/QED_VALIDITY_RESULTS.pass6.md#L49

Relevant Code

```
let round_trip = (deposit_rate_e9 as u128)
    .checked_mul(redeem_rate_e9 as u128)
    .ok_or(ChanceryError::ArithmeticOverflow)?;

if round_trip > (RATE_PRECISION_E9 as u128) * (RATE_PRECISION_E9 as u128) {
    return Err(ChanceryError::AssetEconomicsInvariantViolated.into());
}
```

```
pub deposit_rate_e9: Option<u64>,
pub redeem_rate_e9: Option<u64>,
```

```
apply_asset_config_update(&mut proposed, args);
```

```
AssetConfig::assert_economic_invariants(proposed.deposit_rate_e9, proposed.redeem_rate_e9)
    .map_err(|_| ChanceryError::AssetEconomicsInvariantViolated)?;
```

```
let out = (token_amount as u128)
    .checked_mul(self.redeem_rate_e9 as u128)
    .ok_or(ChanceryError::ArithmeticOverflow)?
    .checked_div(RATE_PRECISION_E9 as u128)
    .ok_or(ChanceryError::DivisionByZero)?;
```

Mitigation Suggestion

Enforce the per-leg invariant in `assert_economic_invariants`: `deposit_rate_e9 <= RATE_PRECISION_E9` and `redeem_rate_e9 <= RATE_PRECISION_E9`. Keep the product bound as a defense-in-depth check.

If redeem rates above par are intended, gate them with a supply-aware reserve check and update the formal model to match that behavior.

Remediation

Acknowledged.

Description

`RemoteDomainPolicy` identifies the remote token addresses, but it does not store the remote token decimals, a canonical denomination, or a conversion ratio.

Outbound messages commit a raw local base-unit amount. Inbound consumption then verifies the same raw integer and passes it directly to the local Token-2022 mint CPI. If governance activates a corridor between token representations with different decimals, the same integer can represent different whole-token values on each side.

For example, if the remote token has 9 decimals and the local issued token has 6 decimals, one remote whole token is encoded as `1_000_000_000` base units. `consume_inbound_message` mints that value unchanged on Solana, where it represents 1,000 local whole tokens.

This requires an authorized corridor configuration and valid attestations, so it is not permissionless. However, once configured, an otherwise authorized holder can receive excess local issued tokens and redeem them against available reserves.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/state/remote_domain_policy.rs#L120-L152
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/message_hash.rs#L82-L103
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L682-L727
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/consume_inbound_message.rs#L479-L504
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/consume_inbound_message.rs#L694-L701

Relevant Code

```
pub remote_issued_token: [u8; 32],
```

```
pub remote_asset: [u8; 32],
```

```
pub amount: u128,
```

```
amount: net_output_amount as u128,
```

```
    cpi_burn(  
        issued_token_program_account_info,  
        source_issued_token_account_info,  
        issued_token_mint_account_info,  
        sender_account_info,  
        args.amount,  
    )?;
```

```
amount: args.amount as u128,
```

```
    cpi_mint_to(  
        issued_token_program_account_info,  
        issued_token_mint_account_info,  
        recipient_issued_token_account_info,  
        mint_authority_pda_account_info,  
        mint_authority_b,  
        args.amount,  
    )?;
```

Mitigation Suggestion

Store the remote token decimals or an explicit conversion ratio in **RemoteDomainPolicy**. Convert cross-chain amounts into a canonical denomination before hashing and before minting or burning. Reject corridors where the denomination mapping is not explicitly configured.

Remediation

Acknowledged.

Description

Authority-transfer proposals bind their governance epoch only by storing the proposing governance **Pubkey**. A governance rotation from A to B invalidates A's outstanding proposals, but rotating from B back to A makes those proposals valid again because acceptance only compares the stored key to the current governance key.

Each role has a separate **AuthorityTransfer** PDA. Therefore, an emergency-authority proposal can remain pending while the governance role rotates A to B and then back to A. If the stale proposal is still inside its acceptance window, the stale nominee can accept it without approval from the latest governance epoch.

The stale nominee can then acquire emergency authority and pause settlement. No funds are stolen, and exploitation requires two legitimate governance rotations back to the original key, so the issue is low severity.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/core/state/authority_transfer.rs#L51-L92
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/core/instructions/propose_authority_transfer.rs#L126-L143
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/core/instructions/propose_authority_transfer.rs#L210-L220
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/core/instructions/accept_authority_transfer.rs#L101-L107
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/core/instructions/accept_authority_transfer.rs#L126-L131
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/control/instructions/set_global_pause.rs#L84-L95

Relevant Code

```
/// One PDA per role kind allows each role to have an independent in-flight  
/// transfer without contention.
```

```
/// Governance authority that created this proposal. Acceptance requires  
/// this to still be the live governance authority, so every outstanding  
/// proposal is automatically void the moment governance rotates.  
pub proposing_governance: Pubkey,
```

```
let (expected_key, bump) = Pubkey::find_program_address(  
    &[seeds::AUTHORITY_TRANSFER, &[args.role_kind]],  
    &program_id,  
);
```

```
rec.proposing_governance = chancery_config.governance_authority;  
rec.expires_at_slot      = expires_at_slot;
```

```
if proposing_governance != chancery_config.governance_authority {  
    return Err(ChanceryError::AuthorityTransferProposerSuperseded.into());  
}
```

```
authority_role::EMERGENCY => chancery_config.emergency_authority = new_authority,
```

```
let is_emergency = authority_account_info.key == &chancery_config.emergency_authority;  
if !args.is_clear && !(is_governance || is_ops || is_emergency) {  
  return Err(ChanceryError::InsufficientRole.into());  
}
```

Mitigation Suggestion

Track a monotonically increasing governance generation and bind each authority-transfer proposal to it. Increment the generation whenever governance changes and reject proposals from earlier generations.

Remediation

Acknowledged.

Description

`emit_outbound_message` only requires a nonzero `expires_at_unix_timestamp` to be greater than the current timestamp. The remote-domain policy also enforces only a maximum expiry window. There is no minimum message lifetime, so `expires_at_unix_timestamp = now + 1` is accepted.

Each accepted outbound emission burns the sender's principal, consumes the next outbound nonce, and emits a cross-chain message with that short expiry. Such a message can expire before a realistic finality, attestation, and relay cycle can deliver it.

Alice, an authorized sender with `CAN_EMIT_OUTBOUND_MESSAGE` at pathway scope, can repeatedly emit dust outbound messages with `expires_at_unix_timestamp = clock.unix_timestamp + 1`. Each message consumes a sequential outbound nonce. If the remote domain enforces strict inbound ordering, it must retire each expired message before any later Solana-to-remote transfer can be delivered.

Volume caps do not meaningfully limit dust emissions. Only `maximum_actions_per_hour` or `maximum_actions_per_day` can limit the number of such messages. Alice's cost is transaction fees, protocol fees, and temporarily locked principal. The burned principal can be recovered through `reclaim_expired_outbound`, but only after the remote side retires and attests to the expired message.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L276-L279
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L380-L389
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/state/remote_domain_policy.rs#L47-L62
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/state/remote_domain_policy.rs#L387-L401
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L434-L442
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L721-L727

Relevant Code

```
if args.expires_at_unix_timestamp != 0 && args.expires_at_unix_timestamp <= clock.unix_timestamp {  
    return Err(ChanceryError::MessageExpired.into());  
}
```

```
remote_policy.assert_expiry_within_window(  
    args.expires_at_unix_timestamp,  
    clock.unix_timestamp,  
)?;
```

```
let max_expiry = i128::from(now) + i128::from(message_expiry_seconds);  
i128::from(expires_at_unix_timestamp) > max_expiry
```

```
let source_nonce = {  
    let mut n = RemoteNonce::load_mut_for_verified_pda(  

```

```
        remote_nonce_account_info,  
        args.remote_domain_id,  
        &nonce_scope_key,  
        remote_nonce_bump,  
    )?;  
  
    n.take_outbound()?;  
};
```

```
cpi_burn(  
    issued_token_program_account_info,  
    source_issued_token_account_info,  
    issued_token_mint_account_info,  
    sender_account_info,  
    args.amount,  
)?;
```

Mitigation Suggestion

Add `minimum_message_lifetime_seconds` to `RemoteDomainPolicy` and require `expires_at_unix_timestamp >= now + minimum_message_lifetime_seconds` in `emit_outbound_message`. Classify a decrease in this minimum lifetime as widening.

Remediation

Acknowledged.

Description

Counterparty and executor daily **UsageWindow** accounts are derived only from the dimension type and party address. The derivation does not include the asset mint or the selected **LimitPolicy**, so the same party shares one daily accumulator across all assets and pathways.

Settlement handlers then load the current pathway's counterparty or executor policy, read the shared party window, add the current asset's raw base units, and compare the result against the current policy's raw-unit cap.

For example, if Asset A has 9 decimals and permits **1_000_000_000** units, while Asset B has 6 decimals and permits **1_000_000** units, a successful Asset A settlement records **1_000_000_000** in the party's shared **gross_in**. A later Asset B settlement is checked as **1_000_000_000 + asset_b_amount > 1_000_000**, so it returns **DailyLimitBreached** even though the party has not used Asset B.

This affects both counterparty and executor dimensions. Mint and redeem totals are stored separately as **gross_in** and **gross_out**, so inflows do not contaminate outflows. The collision occurs between same-direction settlements across assets.

Alice, an authorized delegated executor, can execute a valid settlement for Bob, the principal, on one asset and contaminate Bob's shared counterparty window. Bob's later principal-signed or delegated transactions through another valid pathway can then fail despite being below that pathway's own limit.

There is no direct value loss because the contaminating settlement executes normally and later transactions revert atomically. However, valid mint and redeem functionality can become unavailable for up to one day and can be repeated while the executor retains the required authorization.

Location

- <https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/limits/dimension.rs#L45-L49>
- <https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/limits/dimension.rs#L124-L155>
- <https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/limits/dimension.rs#L161-L170>
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/limits/state/limit_policy.rs#L397-L402
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/settlement/instructions/mint_direct.rs#L538-L562
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/settlement/instructions/mint_delegated.rs#L639-L692
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/settlement/instructions/mint_delegated.rs#L877-L888

Relevant Code

```
/// sha256(scope_kind || party_key) - the usage-window seed hash for a
/// dimension. For ASSET this equals `LimitPolicy::scope_hash()` of the asset
/// policy; for template dimensions it is keyed by the concrete party.
pub fn dimension_window_scope_hash(scope_kind: u8, party_key: &Pubkey) -> [u8; 32] {
    hashv(&[&[scope_kind], party_key.as_ref()]).to_bytes()
}
```

```

pub fn counterparty_dimension_binding() -> (u8, Pubkey) {
    (scope::COUNTERPARTY, Pubkey::default())
}

pub fn executor_dimension_binding() -> (u8, Pubkey) {
    (scope::EXECUTOR, Pubkey::default())
}

let current_gross = if is_inflow {
    prepared.gross_in
} else {
    prepared.gross_out
};

policy.assert_window_volume(current_gross, amount, ChanceryError::DailyLimitBreached?);

let new_total: u128 = window_gross
    .checked_add(additional as u128)
    .ok_or(ChanceryError::ArithmeticOverflow)?;

if new_total > cap as u128 {
    return Err(error.into());
}

let (cp_scope_kind, cp_scope_key) = counterparty_dimension_binding();

principal_account_info.key,
asset_amount,
true,

let (exec_scope_kind, exec_scope_key) = executor_dimension_binding();

executor_account_info.key,
asset_amount,
true,

```

Mitigation Suggestion

Do not aggregate unrelated asset base units in one party window. Make counterparty and executor windows asset-specific, such as `sha256(scope_kind || asset_mint || party_key)`, and bind the corresponding policies to that asset. If separate policy references are intended to maintain separate budgets, include a stable policy dimension without allowing policy replacement to reset accumulated usage.

Remediation

Acknowledged.

ACC-L10 Fee Policies Can Be Valid but Unexecutable for Every Settlement Amount

LOW

ACKNOWLEDGED

Description

`FeePolicy::assert_parameter_sanity` validates field shapes, denomination consistency, fee and rebate exclusivity, caps, supported flags, and recipient configuration. It does not prove that the configured arithmetic can succeed for any positive settlement output.

For example, a matching-denomination policy with a flat fee of `100`, a flat rebate of `200`, `REBATE_TO_PRINCIPAL`, and `net_fee_floor_zero = false` can pass registration and update validation. At execution, the assessed fee is always `100` and the nominal rebate is always `200`, so every computation returns `FeePolicyNegativeNetFeeNotAllowed`, regardless of the settlement amount.

Pathway reference validation also reruns only `assert_parameter_sanity`, so an active pathway can reference the unusable policy.

This does not directly cause principal loss because settlement transactions roll back atomically. However, every settlement in the policy's matching direction reverts until the policy or pathway is reconfigured, leaving the active pathway unavailable in practice.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/fees/state/fee_policy.rs#L309-L379
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/fees/instructions/register_fee_policy.rs#L120-L142
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/fees/instructions/update_fee_policy.rs#L220-L224
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/pathway/reference_validation.rs#L125-L134
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/fees/state/fee_policy.rs#L517-L566

Relevant Code

```
pub fn assert_parameter_sanity(&self) -> Result<(), ProgramError> {
    if self.fee_policy_flags & !SUPPORTED_FEE_FLAGS != 0 {
        || self.net_fee_floor_zero > 1
        || self.percent_fee_bps > 10_000
        || self.rebate_bps > 10_000
    }
    {
        return Err(ChanceryError::FeePolicyInvalidParameters.into());
    }
}
```

```
if self.rebate_flat_amount != 0 && self.rebate_bps != 0 {
    return Err(ChanceryError::FeePolicyInvalidParameters.into());
}

let has_rebate = self.rebate_flat_amount != 0 || self.rebate_bps != 0;
let rebate_to_principal =
    self.fee_policy_flags & fee_flag::REBATE_TO_PRINCIPAL != 0;
```

```
p.assert_parameter_sanity()?;
p.assert_denomination_consistent()?;
```

```
proposed.assert_parameter_sanity()?;
proposed.assert_denomination_consistent()?;
assert_fee_policy_denomination_unchanged(&current, &proposed)?;
```

```
fee_policy.assert_parameter_sanity()?;
```

```
let effective_rebate = if self.net_fee_floor_zero != 0 {
    nominal_rebate.min(assessed_fee)
} else {
    if nominal_rebate > assessed_fee {
        return Err(ChanceryError::FeePolicyNegativeNetFeeNotAllowed.into());
    }
    nominal_rebate
};
```

Mitigation Suggestion

Extend fee-policy validation to prove that at least one positive output can complete the configured arithmetic. Reject flat-rebate schedules whose rebate necessarily exceeds the maximum attainable assessed fee while `net_fee_floor_zero` is disabled. Keep separate pathway validation against the asset's actual output range.

Remediation

Acknowledged.

FIXED

```

        freeze_rec_account_info,
        issued_token_account_info.key,
        rec_bump,
    )?;

    record.status          = crate::constants::basic_freeze_status::FROZEN;
    record.reason_code     = args.reason_code;
    record.frozen_by       = *authority_account_info.key;
    record.frozen_at_slot  = clock.slot;
    record.freeze_flags    = args.freeze_flags;
}

```

```

pub(super) fn cpi_freeze_account<'a>(
    token_program_account_info: &AccountInfo<'a>,
    token_account_info:         &AccountInfo<'a>,
    mint_account_info:          &AccountInfo<'a>,
    freeze_authority_account_info: &AccountInfo<'a>,
    freeze_authority_bump:       u8,
) -> ProgramResult {

```

Mitigation Suggestion

When `record_disposition` is `Update`, verify that the Token-2022 account state is frozen, skip the redundant `FreezeAccount` CPI, update the freeze record, and emit evidence. Alternatively, add a dedicated metadata-update instruction for an already-frozen account.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/79>.

Activating a dormant collateral transfer hook ACC-L12 freezes the extension observation and the asset reserve

LOW

ACKNOWLEDGED

Description

We found that activating a previously dormant TransferHook on a collateral mint permanently freezes both that asset's reserve and its extension observation, and that the failure mode differs from the one the code documents.

`extensions.rs:116-117` states that enabling a dormant hook later "changes the observed mask to `TRANSFER_HOOK` and settlement fails closed until Chancery is upgraded". The observed mask never changes. `refresh_asset_extension_observation.rs` calls `assert_observed_mask_allowed` at line 94 and only assigns `cfg.observed_extension_mask` at line 106, so once the mint presents the active `TRANSFER_HOOK` bit the refresh aborts before persisting anything and the previous `DORMANT_TRANSFER_HOOK` value remains. `update_asset_config` rejects any write to that field with `ImmutableFieldChange`, and `refresh_asset_extension_observation` is the only other writer, so the stale value is final

Governance cannot clear the condition either. `assert_observed_mask_allowed` computes the effective forbidden mask as the asset's configured mask OR'd with the compile-time

`collateral_default_forbidden:MINT_FORBIDDEN_LO`, which is `TRANSFER_HOOK`. Removing the bit from `forbidden_extension_mask` therefore has no effect, and no configuration change can make the new observation acceptable

The practical result is that Chancery's own extension checks continue to pass on stale data.

`assert_extensions_approved` sees the approved `DORMANT_TRANSFER_HOOK` value and succeeds, so all seven fund-movement paths – the six settlement handlers and `withdraw_reserve` proceed to their token CPI and fail there instead, because `token_transfer_checked_ix` builds a fixed four-account `TransferChecked` with no `ExtraAccountMetaList` resolution, which Token-2022 requires once a hook is active. `withdraw_reserve` is the only instruction that can move collateral out of the reserve for any other reason, and it is blocked on the same CPI, so the asset's entire reserve balance is unreachable. Recovery requires a program upgrade, as the code comment concedes

The more consequential difference from the documented behaviour is detectability. `spec/09` records `max_extension_observation_age_slots == 0` as a deliberate long-lived-observation sentinel and states that the current production asset profile uses it, so the freshness check never fires. `AssetConfig` therefore reports an approved, in-date, apparently healthy `DORMANT_TRANSFER_HOOK` observation indefinitely for an asset whose reserve is already frozen and whose collateral has become untransferable. Operators and monitoring reading asset state see no signal at all, rather than the loud fail-closed rejection the comment describes

The precondition is a mint whose TransferHook program id is later set by its own authority.

`extensions.rs:119-125` explicitly accepts that authority-bearing collateral extensions are outside Chancery's control and are governed only by the requirement that the live observation be included in the approved mask. That requirement is what silently stops holding here

Location

<https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/constants/extensions.rs#L111-L131>

https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/issuance/instructions/refresh_asset_extension_observation.rs#L88-L107

https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/core/state/asset_config.rs#L334-L348

https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/core/instructions/update_asset_config.rs#L113-L119

Relevant Code

```
// DORMANT_TRANSFER_HOOK and is allowed; enabling it later changes the observed
// mask to TRANSFER_HOOK and settlement fails closed until Chancery is upgraded.

let mint_data = asset_mint_account_info.try_borrow_data()?;
let observed = parse_mint_extension_mask_full(&mint_data)?;

if &cfg.asset_mint != &chancery_config.issued_token_mint {
    cfg.assert_observed_mask_allowed(observed)?;
} else {

let clock = Clock::get()?;
let old_observed = cfg.observed_extension_mask;

cfg.observed_extension_mask = observed;

pub fn assert_observed_mask_allowed(
    &self,
    observed: [u64; 2],
) -> Result<(), ProgramError> {
    let effective_forbidden = [
        self.forbidden_extension_mask[0] | collateral_default_forbidden::MINT_FORBIDDEN_LO,
        self.forbidden_extension_mask[1] | collateral_default_forbidden::MINT_FORBIDDEN_HI,
    ];

    Self::assert_mask_subset_of_approved(
        observed,
        self.approved_extension_mask,
        effective_forbidden,
    )
}

if args.observed_extension_mask.is_some() {
    return Err(ChanceryError::ImmutableFieldChange);
}
```

Mitigation Suggestion

Separate detection from enforcement so that a hostile collateral state becomes visible rather than invisible.

`refresh_asset_extension_observation` should persist the newly parsed mask and its slot unconditionally, and move the approved and forbidden mask assertion to the consumers that already call

`assert_extensions_approved`, so the recorded observation always reflects the mint's real state and the asset stops reporting an approved posture it no longer has. Emitting a distinct event when a refresh records a mask that violates the effective forbidden set would give operators the signal that is currently absent. Independently, the comment in `extensions.rs` should be corrected, because it describes a state transition the code prevents, and the reserve-side consequence deserves mention: `withdraw_reserve` is blocked on the same CPI as settlement, so the documented "settlement fails closed" understates the effect to a full reserve freeze recoverable only by program upgrade

Remediation

Acknowledged

Description

We found that ten `PermissionRecord` role bits are treated throughout the permission subsystem as real capabilities, but no instruction handler ever consults them. Five of them `CAN_CONFIGURE_ASSET`, `CAN_SET_PATHWAY_POLICY`, `CAN_SET_LIMIT_POLICY`, `CAN_SET_EVIDENCE_POLICY` and `CAN_SET_FEE_POLICY` are listed in `NON_ADMIN_GRANTABLE_ROLE_MASK`, so a narrow non-admin grantor may redistribute them as scoped delegated authority. The other five `CAN_SET_GLOBAL_PAUSE`, `CAN_SET_ASSET_PAUSE`, `CAN_PROPOSE_AUTHORITY_TRANSFER`, `CAN_ACCEPT_AUTHORITY_TRANSFER` and `CAN_SET_MODULE_STATUS` are folded into `DANGEROUS_PERMISSION_ROLE_MASK`, so granting one requires narrow scope, finite expiry and the full `Dangerous` pending-change ceremony with its 24-hour timelock.

The handlers whose names correspond to these bits gate exclusively on the flat `ChanceryConfig.governance_authority`, `operations_authority` and `emergency_authority` singleton keys, with no `PermissionRecord` lookup at all. This applies to `update_asset_config.rs`, `register_pathway_policy.rs`, the limits, evidence and fees instruction modules, `set_global_pause.rs`, `set_asset_pause.rs`, `set_module_status.rs`, `propose_authority_transfer.rs` and `accept_authority_transfer.rs`. A whole-tree search for each of the ten bits returns matches only in `constants/role.rs`, the classification lists in `dangerous_roles.rs`, and unit tests of those masks never in an authorization branch. By contrast `CAN_GRANT_PERMISSION` and `CAN_REVOKE_PERMISSION`, from the same mask family, are consulted throughout `permissions/mutation.rs`.

The gap fails safe rather than open: a subject holding only one of these bits has no more power than an unrelated keypair. The operational consequence is that governance can run a Dangerous-tier, timelocked ceremony to grant, for example, `CAN_SET_GLOBAL_PAUSE` to a named incident-response delegate, reasonably conclude that the delegate can now call `set_global_pause`, and discover during a live incident that the call reverts because the handler never reads the record.

There is also a latent activation risk. `assert_role_schema` rejects a persisted record only when its bits fall outside `ACTIVE_ROLE_MASK`, and `ACTIVE_ROLE_MASK` excludes only retired positions and the later-module bits enumerated in `INACTIVE_RESERVED_ROLE_MASK`. These ten are inside the active mask, so a record granted today persists as valid. If a future release wires any of these handlers to consult the record, every previously issued grant becomes live immediately, with no schema version bump and no new ceremony to re-review who holds it

Location

<https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/constants/role.rs#L85-L126>

https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/permissions/state/permission_record.rs#L249-L259

https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/control/instructions/set_global_pause.rs#L84-L95

https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/control/instructions/set_asset_pause.rs#L147-L157

https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/core/instructions/propose_authority_transfer.rs#L94-L96

https://github.com/SolomonLabs/chancery-audits/blob/c1d461a85380029471fee0017967ef4fc3a31c21/programs/chancery/src/modules/core/instructions/update_asset_config.rs#L17

Relevant Code

```

pub const NON_ADMIN_GRANTABLE_ROLE_MASK: u128 =
    CAN_MINT_DIRECT
    | CAN_REDEEM_DIRECT
    | CAN_MINT_DELEGATED
    | CAN_REDEEM_DELEGATED
    | CAN_EXECUTE_SETTLEMENT
    | CAN_USE_TRILATERAL_PATHWAY
    | CAN_CONFIGURE_ASSET
    | CAN_SET_PATHWAY_POLICY
    | CAN_SET_LIMIT_POLICY
    | CAN_SET_EVIDENCE_POLICY
    | CAN_SET_FEE_POLICY;

pub const ACTIVE_ROLE_MASK: u128 =
    DEFINED_ROLE_MASK & !RETIRED_ROLE_MASK & !INACTIVE_RESERVED_ROLE_MASK;

/// Fail closed unless the record was written against this binary's exact
/// role schema and contains only currently active role bits.
pub fn assert_role_schema(&self) -> Result<(), ProgramError> {
    if self.role_schema_version != role::PERMISSION_ROLE_SCHEMA_VERSION {
        return Err(ChanceryError::PermissionRoleSchemaMismatch.into());
    }

    if role_bits_as_u128(self.role_bits) & !role::ACTIVE_ROLE_MASK != 0 {
        return Err(ChanceryError::PermissionUnknownRoleBits.into());
    }

    Ok(())
}

```

Mitigation Suggestion

Decide per bit whether the capability is delegable in this binary. For bits that should be delegable, add the **PermissionRecord** check to the corresponding handler alongside the existing singleton-authority branch, following the pattern already used for **CAN_WITHDRAW_RESERVE** in **withdraw_reserve.rs**, so that a granted bit actually confers the capability. For bits that are not delegable in this release, move them into **INACTIVE_RESERVED_ROLE_MASK** so grants are rejected at mutation time rather than silently persisted, which also removes the latent activation risk and matches the stated purpose of that mask. Either way the outcome to avoid is a Dangerous-tier ceremony that produces a record conferring nothing, and a future release that turns previously issued records live without re-review

Remediation

Fixed in commit: [SolomonLabs/chancery-audits#87](#)

Description

All direct, delegated, and trilateral settlement handlers use the pathway's shared `LimitPolicy` and `UsageWindow` accounts for both mint and redeem operations. However, the two directions account for volume in different raw-token denominations:

- mint checks and records `asset_amount`, denominated in collateral-token raw units;
- redeem checks and records `issued_token_amount`, denominated in issued-token raw units.

The same `per_transaction_maximum`, `per_hour_maximum`, `per_day_maximum`, `per_seven_day_maximum`, and `per_thirty_day_maximum` values are therefore applied to two quantities with potentially different decimal scales and economic values.

For example, consider:

```
Collateral decimals:  9
Issued-token decimals: 6
Economic exchange rate: 1 collateral = 1 issued token

deposit_rate_e9: 1,000,000
redeem_rate_e9:  1,000,000,000,000
pathway daily maximum: 1,000,000,000
```

These rates correctly provide a one-to-one whole-token exchange:

```
Mint:
1,000,000,000 collateral units = 1 collateral
-> 1,000,000 issued units      = 1 issued token

Redeem:
1,000,000 issued units          = 1 issued token
-> 1,000,000,000 collateral units = 1 collateral
```

However, the shared daily maximum has different meanings in each direction:

```
Mint limit:
1,000,000,000 collateral units = 1 collateral

Redeem limit:
1,000,000,000 issued units = 1,000 issued tokens
                          = 1,000 collateral
```

Consequently, a pathway limit configured as `1,000,000,000` permits only one collateral token to enter through minting, but permits up to 1,000 collateral tokens to leave through redemption.

Although mint and redeem use separate `gross_in` and `gross_output_amount` counters, they are stored in the same window and compared against the same numeric policy maximum. The limit does not normalize the amounts into a common denomination.

The window also calculates:

```
net_flow = gross_in - gross_output_amount
```

This subtracts collateral raw units from issued-token raw units. For example, a complete one-token round trip records:

```
gross_in           = 1,000,000,000 collateral units
gross_output_amount = 1,000,000 issued units
net_flow           = 999,000,000
```

even though the reserve's actual net collateral movement is zero.

An operator may configure a pathway limit assuming it is denominated in collateral units, while the redeem handlers interpret the same value as issued-token units. When token decimals or configured rates differ, this can make one direction substantially more permissive than intended.

In the example above, a pathway intended to limit collateral movement to one token per day permits redemption of 1,000 collateral tokens. This weakens per-transaction and fixed-window controls intended to contain reserve outflows during compromised-key events, operational failures, or abnormal market activity.

The issue does not independently create a profitable mint/redeem cycle. Users must possess and burn issued tokens, the reserve must hold sufficient collateral, and the configured economic-rate invariant still applies. Correctly configured asset-denominated redeem limits can also provide separate containment. Nevertheless, the pathway-level control does not reliably enforce the economic limit an operator may expect.

Additionally, the mixed-denomination `net_flow` value is economically meaningless. Any monitoring, evidence processing, or future net-flow cap that relies on this field may report incorrect pathway exposure or enforce limits against an invalid value.

Severity: Low

Location

- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/pathway/state/pathway_policy.rs#L98-L99
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/limits/state/limit_policy.rs#L60-L73
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/limits/state/limit_policy.rs#L364-L405
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/mint_direct.rs#L401-L426
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/mint_direct.rs#L716-L729
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/mint_delegated.rs#L512-L581
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/mint_delegated.rs#L863-L874
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/mint_trilateral.rs#L535-L602
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/mint_trilateral.rs#L886-L898
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_direct.rs#L387-L412
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_direct.rs#L684-L697
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_delegated.rs#L499-L568
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_delegated.rs#L841-L853
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_trilateral.rs#L527-L596
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/settlement/instructions/redeem_trilateral.rs#L879-L891

- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/limits/state/usage_window.rs#L99-L108
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/limits/state/usage_window.rs#L296-L332

Relevant Code

```
/// Limit policy that gates this pathway. Zero = no limit policy.
pub limit_policy_id: [u8; 32],
```

```
/// Hard cap on a single transaction. 0 = disabled.
pub per_transaction_maximum: u64,

/// Fixed UTC-hour gross flow cap. 0 = disabled.
pub per_hour_maximum: u64,

/// Fixed UTC-day gross flow cap. 0 = disabled.
pub per_day_maximum: u64,

/// Fixed 7-day epoch gross flow cap. 0 = disabled.
pub per_seven_day_maximum: u64,

/// Fixed 30-day epoch gross flow cap. 0 = disabled.
pub per_thirty_day_maximum: u64,
```

```
limit_policy.assert_per_tx(args.asset_amount)?;

let hourly_enforced = limit_policy.per_hour_maximum != 0
|| limit_policy.maximum_actions_per_hour != 0;
```

```
limit_policy.assert_window_volume(
    hourly_usage_window.gross_in,
    args.asset_amount,
    ChanceryError::HourlyLimitBreached,
)?;
```

```
if let Some(prepared_window) = hourly_prepared {
    prepared_window.record_inflow(&accounts[HOURLY_USAGE_WINDOW], args.asset_amount)?;
}

if let Some(prepared_window) = daily_prepared {
    prepared_window.record_inflow(&accounts[DAILY_USAGE_WINDOW], args.asset_amount)?;
}

if let Some(prepared_window) = weekly_prepared {
    prepared_window.record_inflow(&accounts[WEEKLY_USAGE_WINDOW], args.asset_amount)?;
}

if let Some(prepared_window) = monthly_prepared {
    prepared_window.record_inflow(&accounts[MONTHLY_USAGE_WINDOW], args.asset_amount)?;
}
```

```
limit_policy.assert_per_tx(args.issued_token_amount)?;

let hourly_enforced = limit_policy.per_hour_maximum != 0
|| limit_policy.maximum_actions_per_hour != 0;
```

```
limit_policy.assert_window_volume(
    hourly_usage_window.gross_out,
    args.issued_token_amount,
    ChanceryError::HourlyLimitBreached,
)?;
```

```

if let Some(prepared_window) = hourly_prepared {
    prepared_window.record_outflow(&accounts[HOURLY_USAGE_WINDOW], args.issued_token_amount)?;
}

if let Some(prepared_window) = daily_prepared {
    prepared_window.record_outflow(&accounts[DAILY_USAGE_WINDOW], args.issued_token_amount)?;
}

if let Some(prepared_window) = weekly_prepared {
    prepared_window.record_outflow(&accounts[WEEKLY_USAGE_WINDOW], args.issued_token_amount)?;
}

if let Some(prepared_window) = monthly_prepared {
    prepared_window.record_outflow(&accounts[MONTHLY_USAGE_WINDOW], args.issued_token_amount)?;
}

```

```

/// Cumulative gross inflow for the current period (two u64 words, u128
/// semantics).
pub gross_in:                [u64; 2],

/// Cumulative gross outflow for the current period.
pub gross_output_amount:     [u64; 2],

/// Net flow = gross_in - gross_output_amount (two u64 words, i128
/// two's-complement).
pub net_flow:                [u64; 2],

```

```

pub fn record_inflow(&mut self, amount: u64) -> Result<(), ProgramError> {
    let next_gross_in = words_to_u128(self.gross_in)
        .checked_add(amount as u128)
        .ok_or(ChanceryError::ArithmeticOverflow)?;
    let next_action_count = self
        .action_count
        .checked_add(1)
        .ok_or(ChanceryError::ArithmeticOverflow)?;
    let next_net_flow = checked_net_flow_words(
        next_gross_in,
        words_to_u128(self.gross_output_amount),
    )?;

    self.gross_in = u128_to_words(next_gross_in);
    self.net_flow = next_net_flow;
    self.action_count = next_action_count;

    Ok(())
}

/// Record a redeem (outflow) event. All fallible calculations complete
/// before mutation for the same all-or-nothing guarantee as `record_inflow`.
pub fn record_outflow(&mut self, amount: u64) -> Result<(), ProgramError> {
    let next_gross_output_amount = words_to_u128(self.gross_output_amount)
        .checked_add(amount as u128)
        .ok_or(ChanceryError::ArithmeticOverflow)?;
    let next_action_count = self
        .action_count
        .checked_add(1)
        .ok_or(ChanceryError::ArithmeticOverflow)?;
    let next_net_flow = checked_net_flow_words(
        words_to_u128(self.gross_in),
        next_gross_output_amount,
    )?;

    self.gross_output_amount = u128_to_words(next_gross_output_amount);
    self.net_flow = next_net_flow;
    self.action_count = next_action_count;

    Ok(())
}

```

Mitigation Suggestion

Normalize pathway accounting and caps into one denomination before checking or recording usage. Use a shared economic value, such as collateral-denominated raw units, for both mint and redeem pathway limits and **net_flow**.

Split pathway **LimitPolicy** and **UsageWindow** accounts by direction if different denominations are required, and make each policy's denomination explicit.

Add tests with different collateral and issued-token decimals to verify that equal economic movement consumes equal pathway capacity in both directions.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/72>.

ACC-L15 Outbound Messages Can Burn Gross Amount Without a Minimum Remote Amount

LOW

FIXED

Description

The `EmitOutboundMessageArgs` struct commits only the gross `amount` to burn. It does not include a minimum acceptable post-fee remote amount.

The `emit_outbound_message` instruction loads the current `FeePolicy` account during execution, computes `net_output_amount`, commits that net amount to the cross-chain message, and then burns the full gross `args.amount`. If a fee-policy update executes before a pending user transaction, the user can receive materially less remote principal than expected.

The fee computation rejects zero net output, but a valid fee policy can leave only one unit of `net_output_amount`. The signed instruction cannot express that this result is unacceptable.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L165-L193
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L576-L647
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/fees/state/fee_policy.rs#L560-L566
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L656-L701
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/cross_chain/instructions/emit_outbound_message.rs#L764-L800
- https://github.com/SolomonLabs/chancery-audits/blob/0452d8f3349889a237f1c70cb2f41d9583c2469e/programs/chancery/src/modules/fees/instructions/update_fee_policy.rs#L82-L105

Relevant Code

```
pub struct EmitOutboundMessageArgs {
    /// Local pathway PDA selector. Must resolve to a `CROSS_CHAIN_REDEEM` pathway.
    pub pathway_id: [u8; 32],

    /// Destination remote chain-family discriminant. `chain_kind::SOLANA` is
    /// valid for another SVM deployment; corridor identity is not inferred from
    /// this discriminant alone.
    pub remote_chain_kind: u8,

    /// Destination remote domain id, BE-encoded into the policy/nonce PDA seeds.
    pub remote_domain_id: u64,

    /// Amount of issued tokens to burn (Solana-side u64; widened to u128 in canon).
    pub amount: u64,

    /// 32-byte recipient on the remote chain (left-padded for 20-byte EVM addrs).
    pub recipient: [u8; 32],

    /// 32-byte remote-asset identifier; informational on outbound, validated
    /// by the daughter contract per its local config.
    pub destination_asset: [u8; 32],

    /// One of `cross_chain_message_kind::OUTBOUND_*`.
    pub message_kind: u8,

    /// Unix timestamp after which the daughter must reject this message.
```

```

    /// 0 = no expiry. Recommended: clock + remote_domain_policy.message_expiry_seconds.
    pub expires_at_unix_timestamp: i64,
}

```

```

let (
    fee_output_amount,
    rebate_amount,
    net_output_amount,
    fee_policy_id,
    route_fee_to_recipient,
): (u64, u64, u64, [u8; 32], bool) =
    if accounts.len() > FEE_POLICY
        && pathway.has_fee_policy()
        && accounts[FEE_POLICY].key != &Pubkey::default()
    {
        let fee_policy_bump = FeePolicy::verify_pda(
            &accounts[FEE_POLICY],
            &pathway.fee_policy_id,
            &program_id,
        )?;

        let fp = FeePolicy::load_for_verified_pda(
            &accounts[FEE_POLICY],
            &pathway.fee_policy_id,
            fee_policy_bump,
        )?;

        fp.assert_effective(clock.unix_timestamp)?;

        let computation = fp.compute_issued_token_fee_with_input(args.amount, args.amount)?;
    }

```

```

let net_output = gross_output
    .checked_sub(net_fee)
    .ok_or(ChanceryError::NetOutputZero)?;

if net_output == 0 {
    return Err(ChanceryError::NetOutputZero.into());
}

```

```

let preimage = MessageHashPreimage {
    message_kind:      args.message_kind,
    source_chain_kind: chain_kind::SOLANA,
    destination_chain_kind: remote_policy.remote_chain_kind,
    source_domain_id:   SOLANA_SELF_DOMAIN_ID,
    destination_domain_id: remote_policy.remote_domain_id,
    source_nonce,
    amount:             net_output_amount as u128,
    expires_at_unix_timestamp: args.expires_at_unix_timestamp,
    source_chancery_contract: &source_chancery_bytes,
    destination_chancery_contract: &remote_policy.remote_chancery_contract,
    source_domain_separator: &chancery_domain_separator,
    destination_domain_separator: &remote_policy.remote_domain_separator,
    source_asset:           &source_asset_bytes,
    destination_asset:      &args.destination_asset,
    source_issued_token:    &source_issued_bytes,
    destination_issued_token: &remote_policy.remote_issued_token,
    sender:                 &sender_bytes,
    recipient:              &args.recipient,
    signer_set_id:          &remote_policy.signer_set_id,
};

```

```

// ■■■ BURN: atomic with this entire ix. Failure here reverts the nonce bump. ■■■
cpi_burn(
    issued_token_program_account_info,
    source_issued_token_account_info,
    issued_token_mint_account_info,
    sender_account_info,
    args.amount,

```

```
)?;
```

```
CrossChainOutboundEmitted {
  sequence_nonce:          seq,
  chancery:                *chancery_config_account_info.key,
  slot:                    clock.slot,
  unix_timestamp:          clock.unix_timestamp,
  message_hash:            message_hash,
  canon_version:           MESSAGE_HASH_CANON_VERSION,
  message_kind:            args.message_kind,
  source_chain_kind:       chain_kind::SOLANA,
  destination_chain_kind:  remote_policy.remote_chain_kind,
  source_domain_id:        SOLANA_SELF_DOMAIN_ID,
  destination_domain_id:   remote_policy.remote_domain_id,
  source_nonce,
  amount:                  net_output_amount as u128,
  expires_at_unix_timestamp: args.expires_at_unix_timestamp,
  source_chancery_contract: source_chancery_bytes,
  destination_chancery_contract: remote_policy.remote_chancery_contract,
  source_domain_separator: chancery_domain_separator,
  destination_domain_separator: remote_policy.remote_domain_separator,
  source_asset:            source_asset_bytes,
  destination_asset:       args.destination_asset,
  source_issued_token:     source_issued_bytes,
  destination_issued_token: remote_policy.remote_issued_token,
  sender:                  sender_bytes,
  recipient:               args.recipient,
  signer_set_id:           remote_policy.signer_set_id,
  pathway_id:              args.pathway_id,
  burned_by:               *sender_account_info.key,
  gross_amount:            args.amount as u128,
  fee_output_amount:       fee_output_amount as u128,
  rebate_amount:           rebate_amount as u128,
  fee_policy_id:           fee_policy_id,
},
```

```
pub(super) fn apply_fee_policy_update(p: &mut FeePolicy, args: &UpdateFeePolicyArgs) {
  if let Some(v) = args.fee_policy_flags {
    p.fee_policy_flags = v;
  }

  if let Some(v) = args.flat_fee_in_asset {
    p.flat_fee_in_asset = v;
  }

  if let Some(v) = args.flat_fee_in_issued_token {
    p.flat_fee_in_issued_token = v;
  }

  if let Some(v) = args.percent_fee_bps {
    p.percent_fee_bps = v;
  }

  if let Some(v) = args.fee_cap_amount {
    p.fee_cap_amount = v;
  }

  if let Some(v) = args.minimum_fee_amount {
    p.minimum_fee_amount = v;
  }
}
```

Mitigation Suggestion

Add `minimum_remote_amount` to `EmitOutboundMessageArgs` and require `net_output_amount >= minimum_remote_amount` before computing the message hash and burning `args.amount`.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/71>.

Description

The formal model proves the invariant `issued_token_supply <= total_backing` and models `withdraw_reserveTransition` with a guard requiring `total_backing - amount >= issued_token_supply`.

The deployed `withdraw_reserve` instruction does not enforce an equivalent check.

Location

- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/spec/12_Formal_Verification.md#L29-L35
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/formal_verification/Spec.generated-raw.lean#L139-L142
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/formal_verification/Spec.generated-raw.lean#L273-L275
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/formal_verification/Spec.generated-raw.lean#L737-L807
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/reserve/instructions/withdraw_reserve.rs#L6-L21
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/reserve/instructions/withdraw_reserve.rs#L239-L269
- https://github.com/SolomonLabs/chancery-audits/blob/a1de907b94d24b1665ad40d3a674cc005fd2ebae/programs/chancery/src/modules/reserve/instructions/withdraw_reserve.rs#L277-L327

Relevant Code

```
reserve_asset_balance : Nat
remote_locked_backing : Nat
total_backing : Nat
issued_token_supply : Nat
```

```
def withdraw_reserveTransition (s : State) (signer : Pubkey) (amount : Nat) (within_per_tx_limit : Nat) (within_window_limit : Nat) : Option State :=
  if amount ≤ s.reserve_asset_balance ∧ amount ≤ s.total_backing ∧ s.global_pause_bits = 0 ∧ within_per_tx_limit = 1 ∧ within_window_limit = 1 ∧ s.reserve_destination_registered = 1 ∧ s.reserve_destination_enabled = 1 ∧ s.reserve_destination_asset_matches = 1 ∧ s.reserve_destination_owner_matches = 1 ∧ s.reserve_destination_purpose_valid = 1 ∧ s.reserve_asset_balance ≥ amount ∧ s.total_backing ≥ amount ∧ s.total_backing - amount ≥ s.issued_token_supply ∧ s.event_sequence_nonce + 1 ≤ 18446744073709551615 then
    some { s with reserve_asset_balance := s.reserve_asset_balance - amount, total_backing := s.total_backing - amount, event_sequence_nonce := s.event_sequence_nonce + 1 }
```

```
def settlement_backing_conserved (s : State) : Prop := s.issued_token_supply ≤ s.total_backing
```

```
withdrawal_limit_policy
  .assert_reserve_withdrawal_caps(reserve_destination_account_info.key)?;
withdrawal_limit_policy.assert_per_tx(args.amount)?;
```

```
withdrawal_limit_policy.assert_window_volume(
  withdrawal_daily_usage.gross_out,
  args.amount,
```

```
ChanceryError::DailyLimitBreached,  
)?;
```

```
let actual_destination_amount = cpi_transfer_asset_out(  
  asset_token_program_account_info,  
  reserve_asset_token_account_info,  
  asset_mint_account_info,  
  destination_asset_token_account_info,  
  reserve_authority_account_info,  
  reserve_authority_bump,  
  args.amount,  
)?;
```

Mitigation Suggestion

Align the formal model and documentation with the deployed implementation.

Remediation

Fixed in <https://github.com/SolomonLabs/chancery-audits/pull/88>.

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:

- Solana-specific vulnerabilities
- Access control issues
- Arithmetic errors and precision loss
- Race conditions and MEV opportunities
- Logic errors and edge cases
- Performance optimization opportunities
- Invariant violations
- Account confusion vulnerabilities
- Authority check omissions
- Token22 implementation risks and SPL-related pitfalls
- Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.